

Abstraction and Abstract Machine

Abstraction

- Abstraction mechanisms play a crucial role in computing because they allow us to deal with the complexity inherent in most computational systems by **isolating** the aspects that are important in a **particular context**.

abstract machine

- Abstract machines allow describing what an implementation of a programming language is, without requiring us to go into the specific details of any particular implementation.
- *Interpreter*
- *Compiler*

The Concepts of Abstract Machine and the Interpreter

- An abstract machine is an abstraction of the concept of a physical computer.
 - A digital computer is a physical machine that executes suitably formalized algorithms so that the machine can “understand” them.
 - For actual execution, algorithms must be appropriately formalised using the constructs provided by a programming language

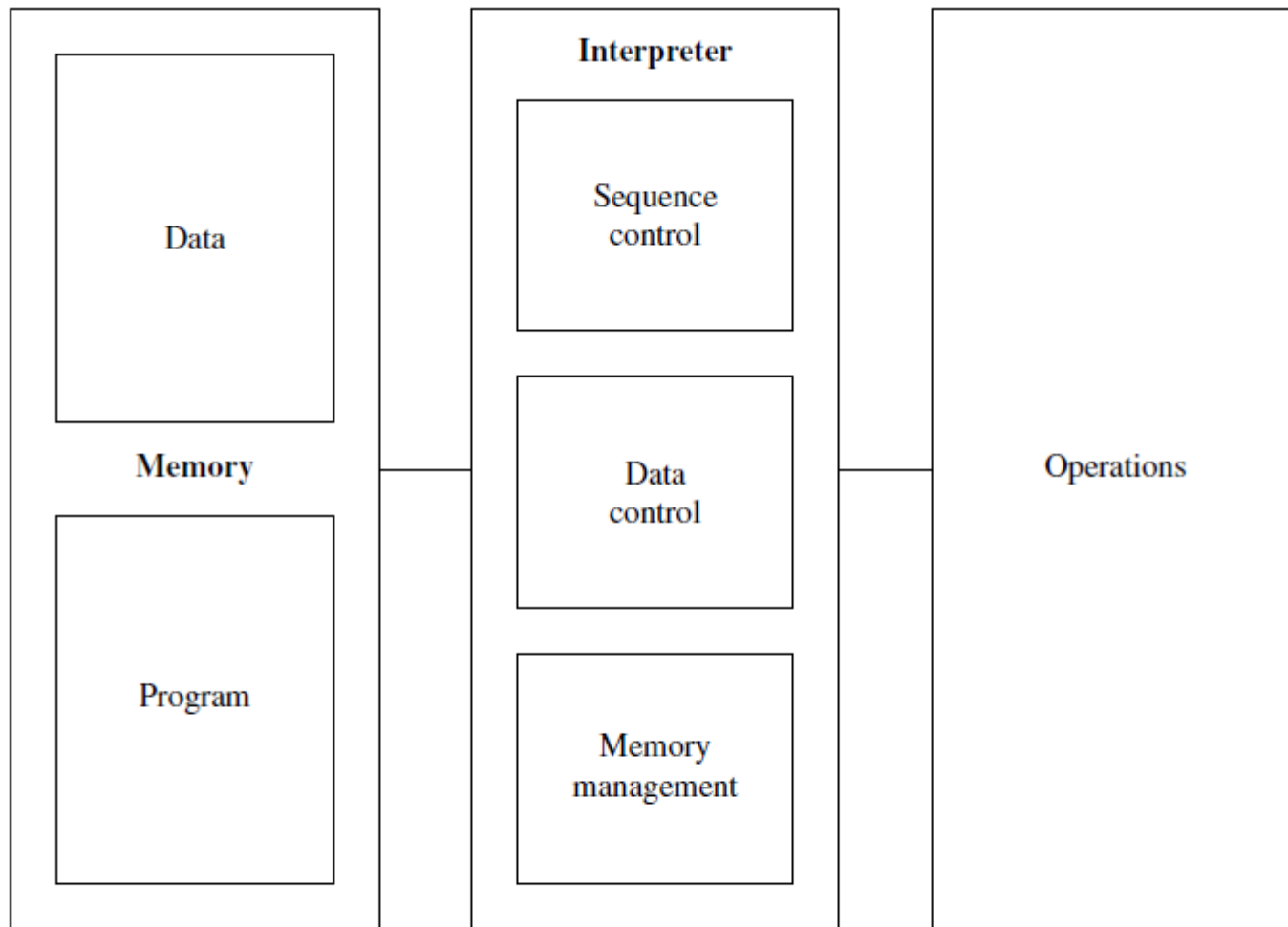


Fig. 1.1 The structure of an abstract machine

- a generic abstract machine ML is composed of a *store* and an *interpreter*. The store serves to save data and programs while the interpreter is the component that executes the instructions of the programs.

- it is sufficient to know that the syntax of L allows us to use a given finite set of constructs, called instructions, to construct programs.
- A *program* in L (or program written in L) is a finite, ordered list of instructions of L .

Definition 1.1 (*Abstract Machine*) Assume that we are given a programming language, $\mathcal{L}$. An *abstract machine* for $\mathcal{L}$, denoted by $\mathcal{M}_{\mathcal{L}}$, is any set of data structures and algorithms which can perform the storage and execution of programs written in $\mathcal{L}$.

The Interpreter

- Any interpreter is specific to its language. However, there are types of operation and an “execution method” common to all interpreters. The type of operation executed by the interpreter and associated data structures, fall into the following categories:
 1. Operations for processing primitive data;
 2. Operations and data structures for controlling the sequence of execution of operations;
 3. Operations and data structures for controlling data transfers;
 4. Operations and data structures for memory management.

“Low-level” and “High-level” Languages

- *low-level*, those languages whose abstract machines are very close to, or coincide with, the physical machine.
- Because the instructions in these languages had to take into account the physical characteristics of the machine, matters that were completely irrelevant to the algorithm had to be considered while writing programs, or in coding algorithms.
- “**machine language**”, we mean the language (a low-level one) of a physical machine.
- *assembly language*, which is a symbolic version of the language of a hardware machine (that is, which uses symbols such as ADD, MUL, etc., instead of their associated hardware binary codes). Programs in assembly language are translated into machine code using a program called an **assembler**.

- *high-level* programming languages are, on the other hand, those which support appropriate abstraction mechanisms to ensure that they are independent of the physical characteristics of the underlying physical computer.
 - suited to expressing algorithms in ways that are relatively easy for the human user to understand.

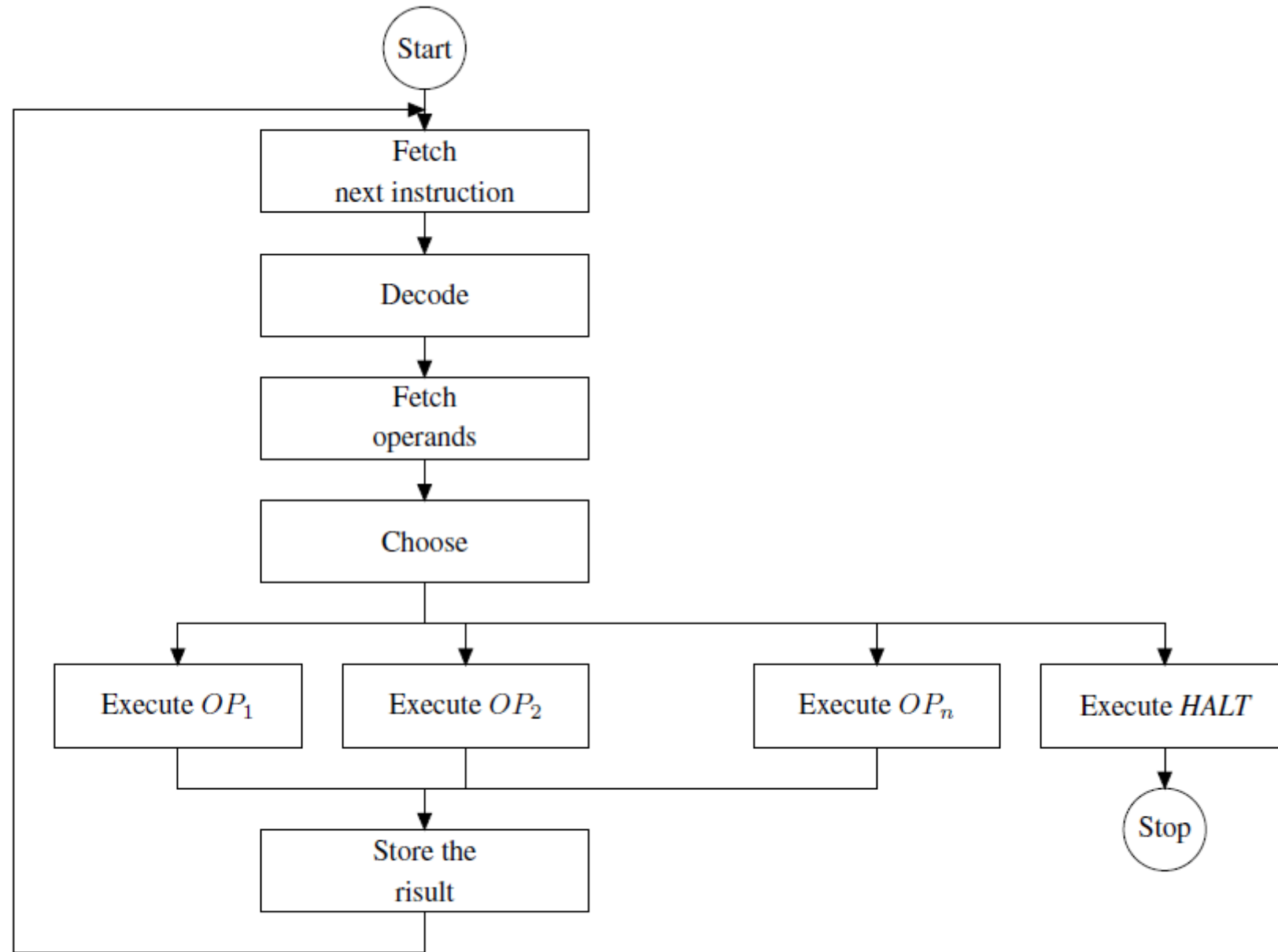


Fig. 1.2 The execution cycle of a generic interpreter

Interpreter

- First, it **fetches** the next instruction to execute from memory.
- The instruction is then **decoded** to determine the operation to be performed as well as its operands.
 - As many operands as required by the instruction are fetched from memory
- the instruction, which must be one of the machine's primitives, is **executed**.
- Once execution of the operation has completed, any results are **stored**.
- The cycle repeats until a halt instruction is reached.

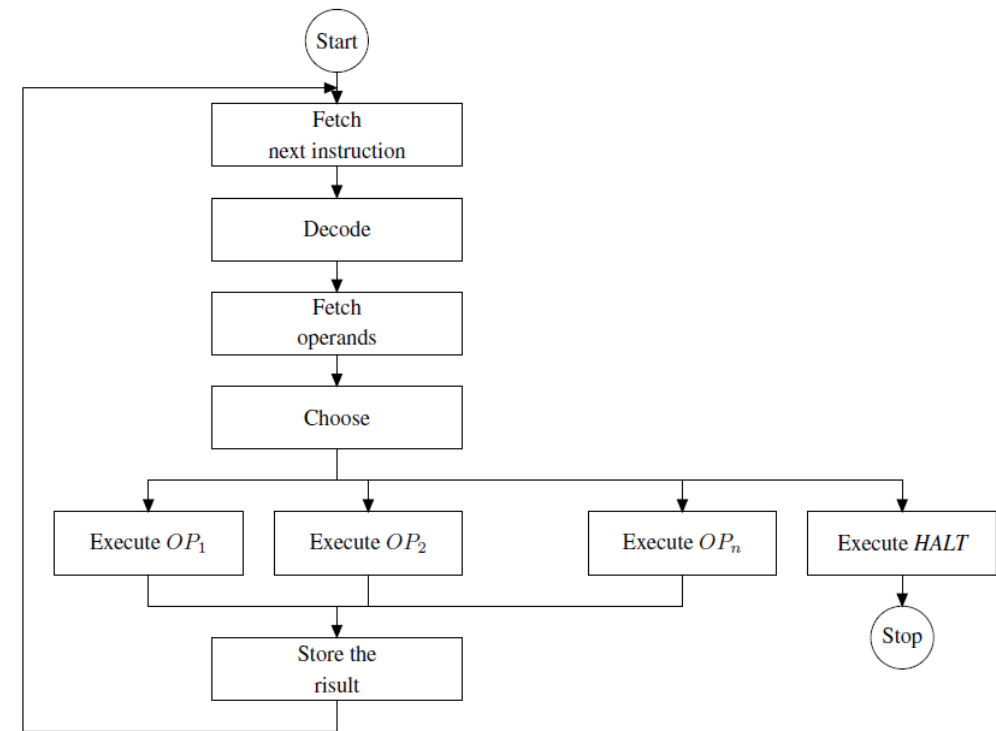


Fig. 1.2 The execution cycle of a generic interpreter

Definition 1.2 (*Machine language*) Given an abstract machine, $\mathcal{M}_{\mathcal{L}}$, the language $\mathcal{L}$ “understood” by $\mathcal{M}_{\mathcal{L}}$ ’s interpreter is called the *machine language* of $\mathcal{M}_{\mathcal{L}}$.

Memory

- The storage component of a physical computer is composed of various levels of memory. Secondary memory implemented using optical or magnetic components; primary memory, organised as a linear sequence of cells, or words, of fixed size (usually a multiple of 8 bits, for example, 32 or 64 bits); cache and the *registers* which are internal to the Central Processing Unit (CPU).
- Physical memory, whether primary, cache, or register file, permits the storage of data and programs. As stated, this is done using the binary alphabet. Data is divided into a few primitive “types”: usually, we have integer numbers, so-called “real” numbers (in reality, a subset of the rationals), characters, and fixed length sequences of bits.

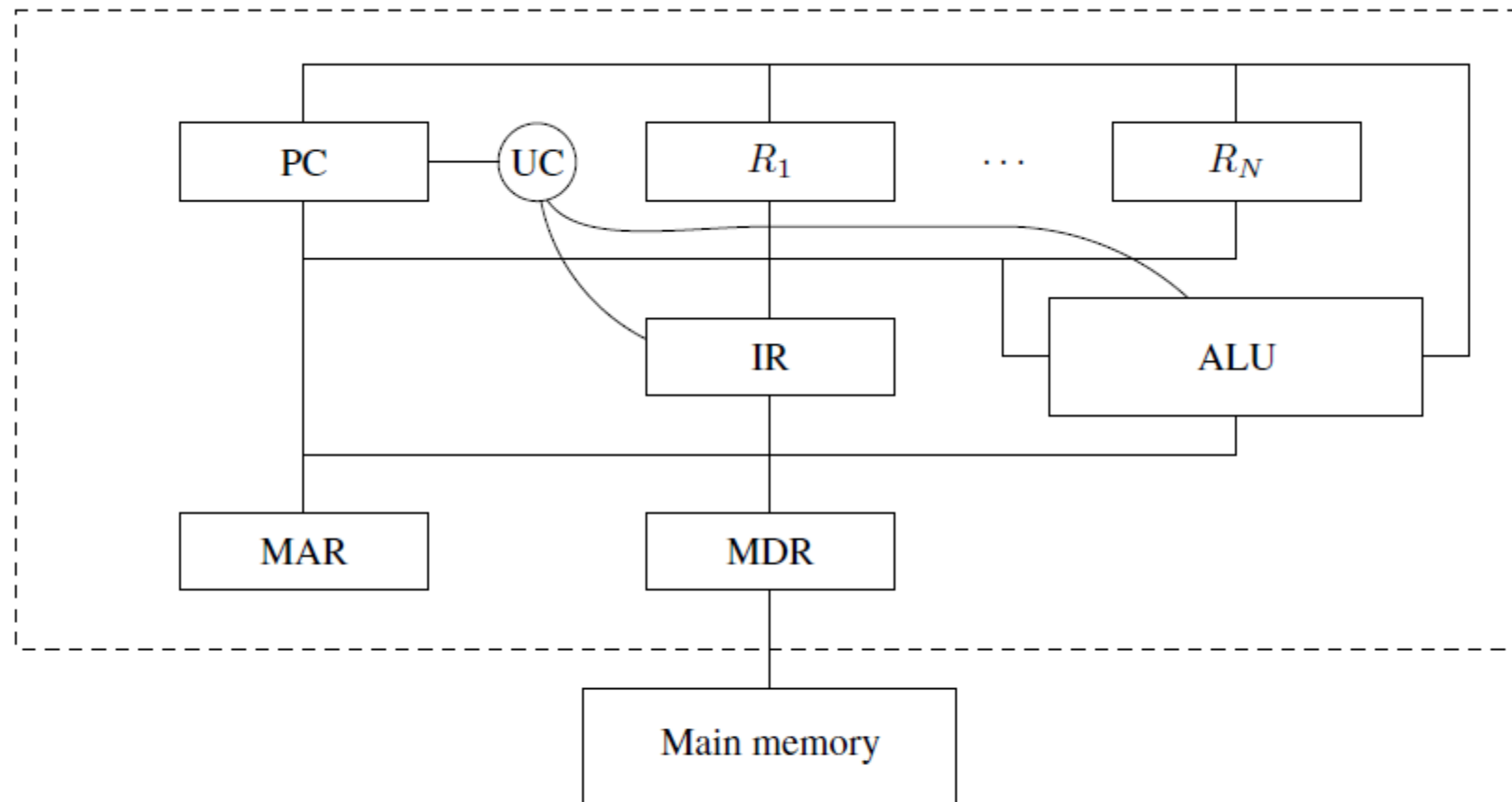


Fig. 1.3 The structure of a conventional computer

The Language of the Physical Machine

- The language, LH which the physical machine executes is composed of relatively simple instructions. A typical instruction with two operands, for example, requires one word of memory and has the format:
- `OpCode Operand1 Operand2`
- where `OpCode` is a unique code that identifies one of the primitive operations defined by the machine's hardware, while `Operand1` and `Operand2` are values that allow the operands to be located by referring to the storage structures of the machine and their addressing modes. For example,
- `ADD R5 , R0`
- might indicate the sum of the contents of registers `R0` and `R5`, with the result being stored in `R5`, while
- `ADD (R5), (R0)`

Interpreter

1. The operations for processing primitive data are the usual arithmetic and logical operations. They are implemented by the ALU (Arithmetic and Logic Unit). Arithmetic operations on integers, and floating-point numbers, booleans are provided, as are shifts, tests, etc.
2. For sequence control, the main data structure is the Program Counter (PC) register, which contains the address of the next instruction to execute. Sequence-control operations specifically use this register and typically include the increment operation (which handles the normal flow of control) and operations that modify the value stored in the PC register (jumps).
3. To handle data transfer, the CPU registers interfacing with the main memory are used. They are: the data address register (the MAR or Memory Address Register) and the data register (MDR or Memory Data Register). There are, in addition, operations that modify the contents of these registers and that implement various addressing modes (direct, indirect, etc.). Finally, there are operations that access and modify the CPU's internal registers.
4. Memory processing depends on the specific architecture. In the simplest case of a register machine that is not multi-programmed, memory management is rudimentary. The program is loaded and immediately starts executing; it remains in memory until it terminates. To increase computation speed, all modern architectures use more sophisticated memory management techniques. In the first place, there are several levels of memory between registers and main memory (i.e., cache memory), whose management needs special data structures and algorithms. Second, some form of multi-programming is almost always implemented (the exe-

- The interpreter for the hardware machine is implemented as a set of physical devices which comprise the **Control Unit** and which support execution of the so called ***fetch-decode-execute cycle***, using the sequence control operations.

- In the *fetch* phase, the next instruction to be executed is retrieved from memory.
- This is the instruction whose address is held in the PC register (the PC register is automatically incremented after the instruction has been fetched).
- The instruction, which is formed of an operation code and perhaps some operands, is then stored in a special register, called the instruction register.

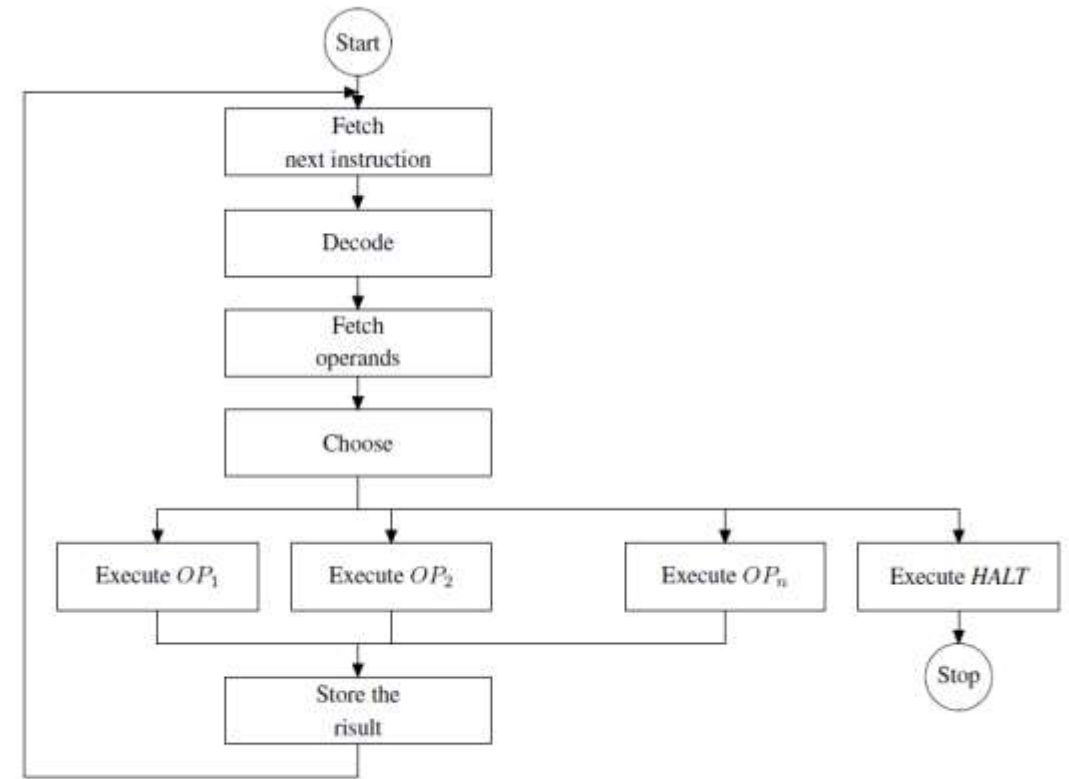


Fig. 1.2 The execution cycle of a generic interpreter

- In the *decode* phase, the instruction stored in the instruction register is decoded using special logic circuits. This allows the correct interpretation of both the instruction's operation code and the addressing modes of its operands. The operands are then retrieved by data transfer operations using the address modes specified in the instruction.

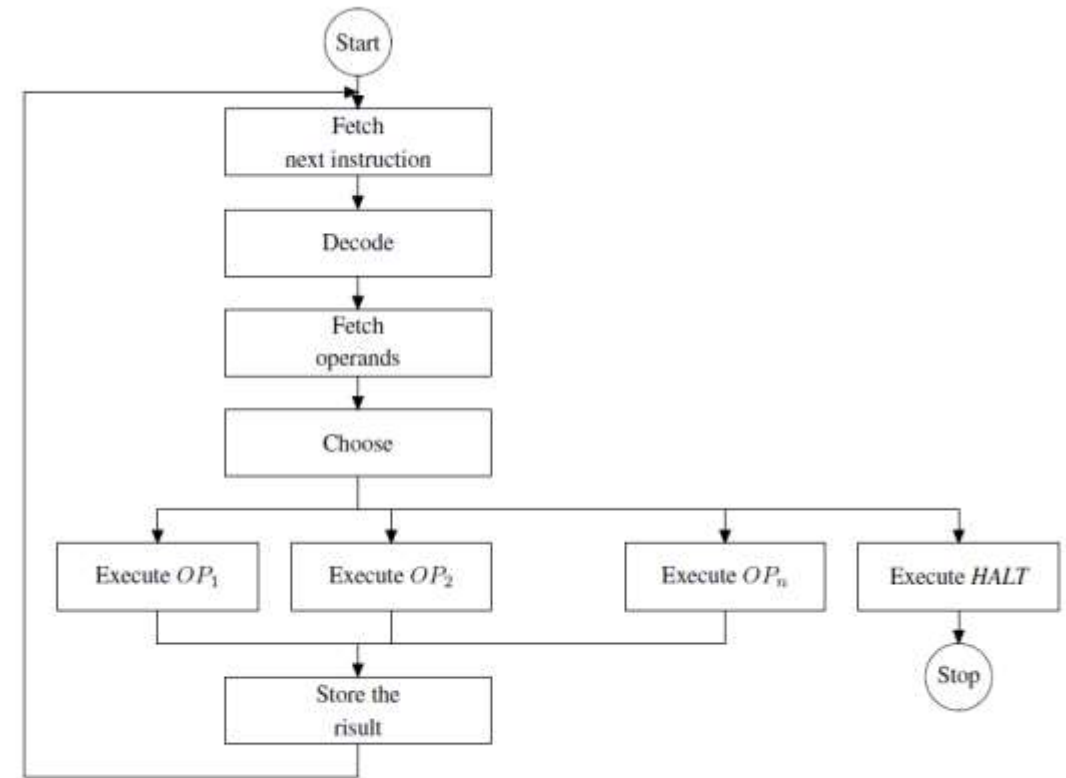


Fig. 1.2 The execution cycle of a generic interpreter

- Finally, in the *execute* phase, the primitive hardware operation is actually executed, for example using the circuits of the ALU if the operation is an arithmetic or logical one. If there is a result, it is stored in the way specified by the addressing mode and the operation code currently held in the instruction register. Storage is performed by means of data-transfer operations. At this point, the instruction's execution is complete and is followed by the next phase, in which the next instruction is fetched and the cycle continues (provided the instruction just executed is not a stop instruction).

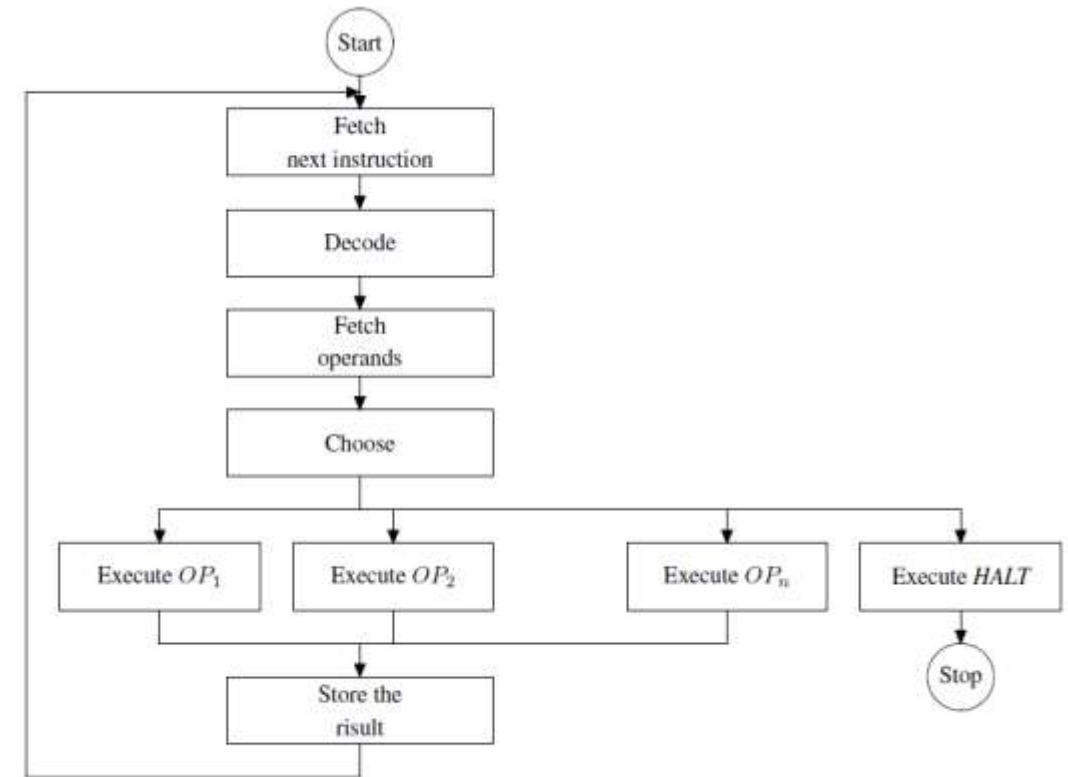


Fig. 1.2 The execution cycle of a generic interpreter

- An abstract machine, ML , is by definition a device which allows the execution of programs written in L . An abstract machine therefore corresponds uniquely to a language, its *machine language*. Conversely, given a programming language, L , there are several (an infinite number of) abstract machines that have L as their machine language. These machines differ from each other in the way in which the interpreter is implemented and in the data structures that they use. They all agree, though, on the language they interpret— L .
- To *implement* a programming language L means realising an abstract machine which has L as its machine language. Before seeing which implementation techniques are used for current programming languages, we first discuss the various theoretical possibilities.

- for implementing an abstract machine to the following three cases and to
- combinations of them:
 - implementation in *hardware*;
 - simulation using *software*;
 - simulation (emulation) using *firmware*.

Partial Functions

A function $f : A \rightarrow B$ is a correspondence between elements of A and elements of B such that, for every element a of A , there exists one and only one element of B , which is denoted by $f(a)$.

A *partial function*, $f : A \rightarrow B$, is also a correspondence between the two sets A and B , but can be undefined for some elements of A . More formally: it is a relation between A and B such that, for every $a \in A$, if there exists a corresponding element $b \in B$, it is unique and is written $f(a)$. The notion of partial function, for us, is important because programs naturally define partial functions. For example, the following program (written in a language with obvious syntax and semantics and whose core will be defined in Fig. 2.11):

```
read(x) ;  
if (x == 1) then print(x) ;  
           else while (true) do skip
```

computes the partial function:

$$f(n) = \begin{cases} 1 & \text{if } n = 1 \\ \text{undefined} & \text{otherwise} \end{cases}$$

The implementation of $\mathcal{L}$ on the host machine $\mathcal{M}_{\mathcal{L}_o}$ takes place using a “translation” from $\mathcal{L}$ to $\mathcal{L}_o$. We can distinguish two conceptually different modes of implementation, depending on whether this translation is “implicit” (implemented by the simulation of $\mathcal{M}_{\mathcal{L}}$ ’s constructs by programs written in $\mathcal{L}_o$) or “explicit”, that is providing for any program in $\mathcal{L}$ its corresponding program in $\mathcal{L}_o$. We will consider these two ways in their ideal forms:

1. *purely interpreted implementation*, and
2. *purely compiled implementation*.

Notation

Below, as we did so far, we use the subscript $\mathcal{L}$ to indicate that a particular construct (machine, interpreter, program, etc.) refers to language $\mathcal{L}$. We will use the superscript $\mathcal{L}$ to indicate that a program is written in language $\mathcal{L}$. We will use $\mathcal{P}rog^{\mathcal{L}}$ to denote the set of all possible programs that can be written in language $\mathcal{L}$, while $\mathcal{D}$ denotes the set of input and output data (and, for simplicity of treatment, we make no distinction between the two).

A program written in $\mathcal{L}$ can be seen as a partial function (see the box):

$$\mathcal{P}^{\mathcal{L}} : \mathcal{D} \rightarrow \mathcal{D}$$

such that

$$\mathcal{P}^{\mathcal{L}}(Input) = Output$$

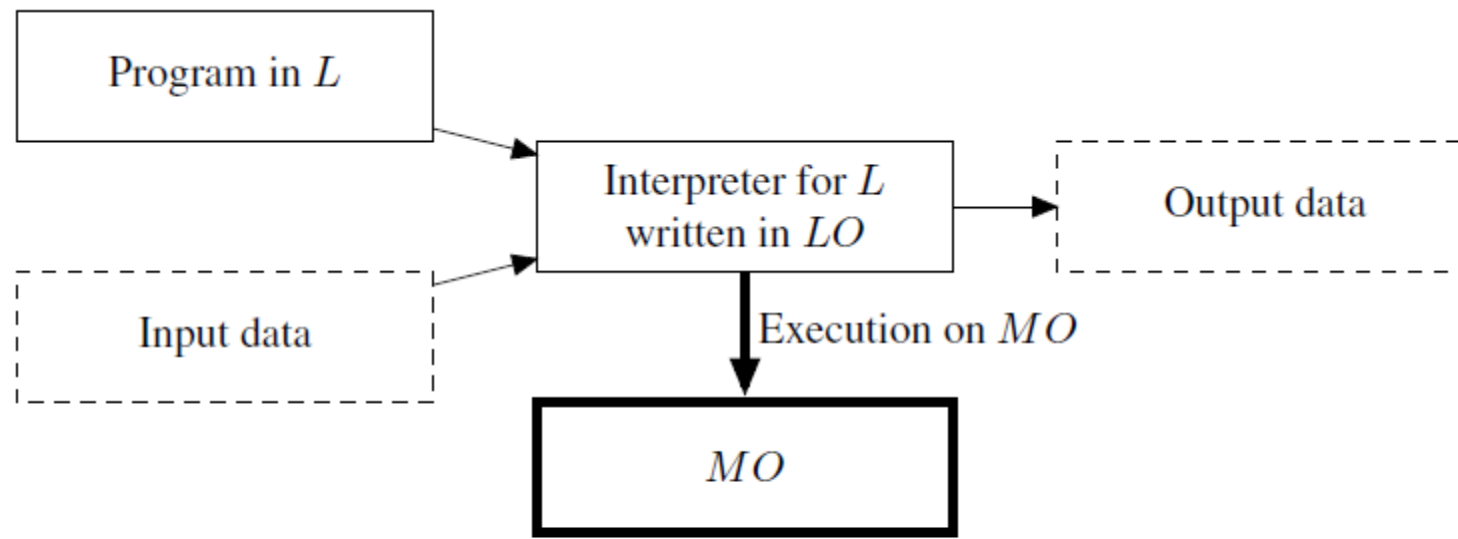


Fig. 1.4 Purely interpreted implementation

- if the execution of PL on input data $Input$ terminates and produces $Output$ as its result. The function is not defined if the execution of PL on its input data, $Input$, does not terminate

- if the execution of PL on input data $Input$ terminates and produces $Output$ as its result. The function is not defined if the execution of PL on its input data, $Input$, does not terminate

Purely Interpreted Implementation

In a *purely interpreted implementation* (shown in Fig. 1.4), the interpreter for $\mathcal{M}_{\mathcal{L}}$ is implemented using a set of instructions in $\mathcal{L}o$. That is, a program is implemented in $\mathcal{L}o$ which interprets all of $\mathcal{L}$'s instructions. This is the *interpreter* of $\mathcal{L}$ (written in $\mathcal{L}o$), and we will write $\mathcal{I}_{\mathcal{L}}^{\mathcal{L}o}$ for it.

Once such interpreter is implemented, executing a program $\mathcal{P}^{\mathcal{L}}$ (written in language $\mathcal{L}$) on specified input data $D \in \mathcal{D}$, we need only to execute the program $\mathcal{I}_{\mathcal{L}}^{\mathcal{L}o}$ on machine $\mathcal{M}_{o_{\mathcal{L}o}}$, with $\mathcal{P}^{\mathcal{L}}$ and D as input data. More precisely, we can give the following definition.

Definition 1.3 (*Interpreter*) An interpreter for language $\mathcal{L}$, written in language $\mathcal{L}o$, is a program which implements a partial function:

$$\mathcal{I}_{\mathcal{L}}^{\mathcal{L}o} : (\mathcal{Prog}^{\mathcal{L}} \times \mathcal{D}) \rightarrow \mathcal{D} \quad \text{such that } \mathcal{I}_{\mathcal{L}}^{\mathcal{L}o}(\mathcal{P}^{\mathcal{L}}, \text{Input}) = \mathcal{P}^{\mathcal{L}}(\text{Input}) \quad (1.1)$$

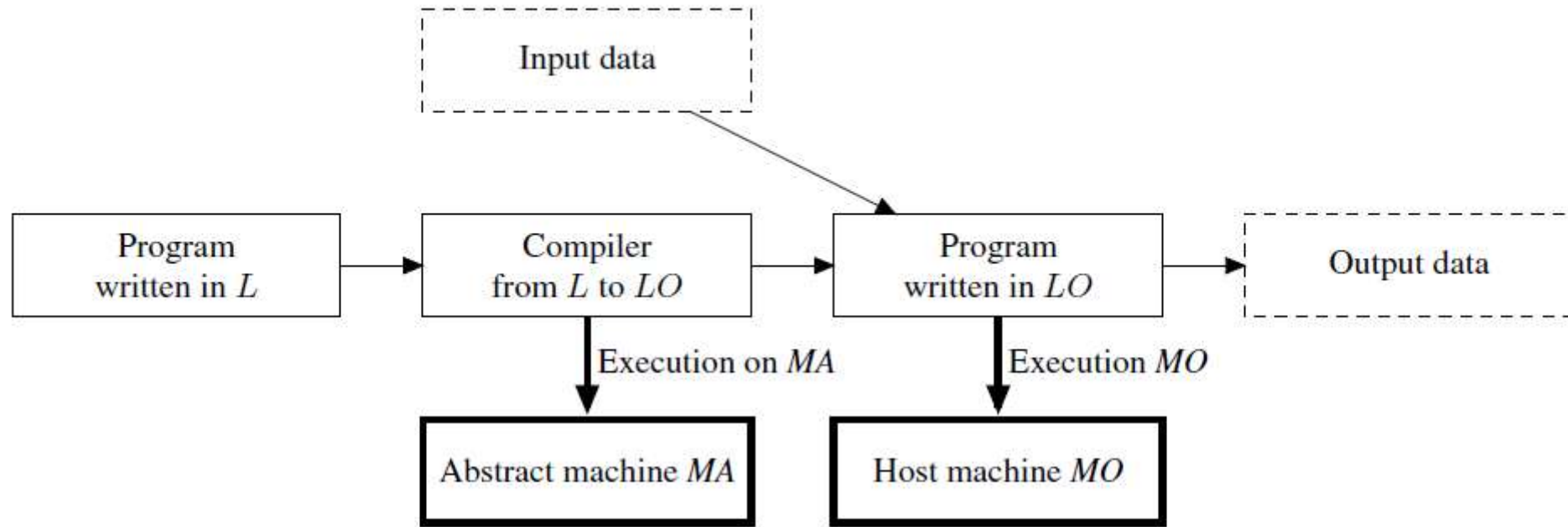


Fig. 1.5 Pure compiled implementation

We have deliberately not specified the nature of the machine $\mathcal{M}_{\mathcal{L}_O}$. The language $\mathcal{L}_O$ can therefore be a high-level language, a low-level language or a firmware one.

Purely Compiled Implementation

With *purely compiled implementation*, as shown in Fig. 1.5, the implementation of $\mathcal{L}$ takes place by explicitly translating programs written in $\mathcal{L}$ to programs written in $\mathcal{L}o$. The translation is performed by a special program called *compiler*; it is denoted by $\mathcal{C}_{\mathcal{L},\mathcal{L}o}$. In this case, the language $\mathcal{L}$ is usually called the *source language*, while language $\mathcal{L}o$ is called the *object language*. To execute a program $\mathcal{P}^{\mathcal{L}}$ (written in language $\mathcal{L}$) on input data D , we must first execute $\mathcal{C}_{\mathcal{L},\mathcal{L}o}$ and give it $\mathcal{P}^{\mathcal{L}}$ as input. This will produce a compiled program $\mathcal{P}^{\mathcal{L}o}$ as its output (written in $\mathcal{L}o$). At this point, we can execute $\mathcal{P}^{\mathcal{L}o}$ on the machine $\mathcal{M}_{\mathcal{L}o}$ supplying it with input data D to obtain the desired result.

Definition 1.4 (*Compiler*) A compiler from $\mathcal{L}$ to $\mathcal{L}o$ is a program which implements a function:

$$\mathcal{C}_{\mathcal{L},\mathcal{L}o} : \text{Prog}^{\mathcal{L}} \rightarrow \text{Prog}^{\mathcal{L}o}$$

such that, given a program $\mathcal{P}^{\mathcal{L}}$, if

$$\mathcal{C}_{\mathcal{L},\mathcal{L}o}(\mathcal{P}^{\mathcal{L}}) = \mathcal{P}^{\mathcal{L}o}, \quad (1.2)$$

then, for every $\text{Input} \in \mathcal{D}^4$:

$$\mathcal{P}^{\mathcal{L}}(\text{Input}) = \mathcal{P}^{\mathcal{L}o}(\text{Input}) \quad (1.3)$$

**purely interpreted vs. purely compiled
implementation**

